Continual Learning

Marcela Londoño y Miguel Ángel Diaz



Aprendizaje Automático



Motivación
Y Relevancia

- El mundo está cambiando constantemente
- Las redes neuronales tradicionales asumen que todos los datos están disponibles a la vez
- En la realidad, los datos llegan en flujo y cambian con el tiempo
- Reentrenar un modelo desde cero con cada actualización es costoso en tiempo, cómputo y dinero, sobre todo con modelos masivos como los LLMs
- Necesitamos modelos que se actualicen sin empezar desde cero

¿Qué es el **Aprendizaje continuo?**

Es un enfoque de aprendizaje de Machine Learning que consiste en entrenar secuencialmente un modelo para nuevas tareas, conservando al mismo tiempo las tareas aprendidas previamente

Los datos llegan como un flujo no estacionario

El numero de tareas no se conoce de antemano

Se inspira en la neuroplasti-cidad del cerebro humano

Ej.: Cuando aprendes a montar bicicleta no olvidas como caminar

¿Qué es el **Olvido catastrófico?**

- Al entrenar en una tarea nueva, los gradientes sobrescriben los pesos que codificaban la anterior
- El modelo pierde casi por completo el conocimiento previo
- **Dilema estabilidad-plasticidad:**

- Plasticidad: aprender información nueva
- Estabilidad: no destruir lo ya aprendido
- Ganar en una suele significar perder en la otra



Naive **Learning**

Entrenar el modelo en la tarea A y, después, continuar entrenando el mismo modelo en la tarea B, sin ningún mecanismo de protección

- Proceso:
  a. Entrenar en A, con buen resultado
  b. Entrenar en B usando solo datos de B
  c. Evaluar de nuevo en A y en B
- ¿Por qué falla? Mientras entrena en B, la red no ve ningún dato de A. Su pérdida solo premia acertar en B, así que los pesos se reajustan solo para B

- ¿Por qué es importante?
  - Es la línea base: lo que ocurre por defecto si no hacemos nada
  - Da el peor caso y permite medir cuánto mejora cada estrategia

Nos muestra el olvido catastrófico en su manera más pura

Experience **Replay**

Guardar una pequeña memoria de ejemplos de tareas anteriores y reutilizarlos cuando se aprende algo nuevo

- Proceso:
  a. Entrenar en A y guardar una muestra de sus ejemplos (la memoria)
  b. Al entrenar en B, mezclar los datos de B con los de la memoria
  c. La red actualiza sus pesos viendo tareas nuevas y viejas a la vez
- ¿Por qué funciona? La pérdida ahora también penaliza equivocarse en A, así que el gradiente ya no destruye lo aprendido

- Ventajas: simple, efectivo, fácil de implementar
- Limitaciones:
  - Hay que almacenar datos, y la memoria crece con más tareas
  - Privacidad: no siempre se pueden guardar datos (por ejemplo, de pacientes)
  - Pocos ejemplos pueden causar sobreajuste a la memoria

Aprende lo nuevo, pero repasa lo anterior para no olvidar

Elastic Weight Consolidation (EWC)

- Idea central: no todos los pesos son igual de importantes para la tarea anterior
  - Pesos críticos para A → deben moverse poco
  - Pesos poco relevantes para A → pueden moverse libremente para aprender B
- No guarda datos, solo guarda:
  - Los pesos óptimos de A (θ*)
  - Un valor de importancia por peso (Fisher)
- Analogía del resorte: cada peso está atado a su valor anterior con un resorte
  - Importante → resorte duro
  - No importante → resorte suave
- Ventaja: no almacena ejemplos (privacidad, memoria)

Inspirado en la consolidación sináptica del cerebro

Elastic Weight Consolidation (EWC)

Función al aprender B:

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i(\theta_i - \theta^*_{A,i})^2$$

- L_B: Pérdida normal de la tarea nueva
- θ^*_i: Valor del peso i tras aprender A
- F_i: Importancia del peso i (diagonal de la matriz de Fisher)
- λ: Fuerza de la protección

Código Naive

Enlace GitHub

```python
def new_model():
    model = create_model()
    model.load_state_dict(copy.deepcopy(INITIAL_STATE))
    return model


naive_model = new_model()

# 1. Estado inicial: antes de aprender ninguna tarea
naive_initial_A = evaluate(naive_model, x_test_A, y_test_A)
naive_initial_B = evaluate(naive_model, x_test_B, y_test_B)

print("=" * 45)
print("RESULTADOS INICIALES")
print("=" * 45)
print(f"Precisión A: {naive_initial_A['accuracy']:.2f}%")
print(f"Precisión B: {naive_initial_B['accuracy']:.2f}%")


# 2. Primera fase: aprender dígitos del 0 al 4
print("\n" + "=" * 45)
print("ENTRENAMIENTO DE LA TAREA A")
print("=" * 45)

train_model(
    naive_model,
    x_train_A,
    y_train_A,
    epochs=EPOCHS,
    lr=LEARNING_RATE
)

naive_after_A = {
    "A": evaluate(naive_model, x_test_A, y_test_A),
    "B": evaluate(naive_model, x_test_B, y_test_B)
}

print("\nResultados después de entrenar A:")
print(f"Precisión A: {naive_after_A['A']['accuracy']:.2f}%")
print(f"Precisión B: {naive_after_A['B']['accuracy']:.2f}%")
```

```python
# 3. Segunda fase: aprender dígitos del 5 al 9 sin datos de A
print("\n" + "=" * 45)
print("ENTRENAMIENTO DE LA TAREA B")
print("=" * 45)

train_model(
    naive_model,
    x_train_B,
    y_train_B,
    epochs=EPOCHS,
    lr=LEARNING_RATE
)

naive_after_B = {
    "A": evaluate(naive_model, x_test_A, y_test_A),
    "B": evaluate(naive_model, x_test_B, y_test_B)
}

print("\nResultados después de entrenar B:")
print(f"Precisión A: {naive_after_B['A']['accuracy']:.2f}%")
print(f"Precisión B: {naive_after_B['B']['accuracy']:.2f}%")


# 4. Resumen final
olvido_A = (
    naive_after_A["A"]["accuracy"]
    - naive_after_B["A"]["accuracy"]
)

precision_promedio = (
    naive_after_B["A"]["accuracy"]
    + naive_after_B["B"]["accuracy"]
) / 2
```

```
============================================
RESULTADOS INICIALES
============================================
Precisión A: 0.74%
Precisión B: 19.46%

============================================
ENTRENAMIENTO DE LA TAREA A
============================================
Época 1/3 - pérdida: 0.6457
Época 2/3 - pérdida: 0.1383
Época 3/3 - pérdida: 0.0943

Resultados después de entrenar A:
Precisión A: 98.54%
Precisión B: 0.00%

============================================
ENTRENAMIENTO DE LA TAREA B
============================================
Época 1/3 - pérdida: 2.2045
Época 2/3 - pérdida: 0.2699
Época 3/3 - pérdida: 0.1659

Resultados después de entrenar B:
...
A después de B:     0.00%
B después de B:     95.47%
Olvido de A:        98.54 puntos porcentuales
Precisión promedio final: 47.74%
```

Código
Experience replay

Enlace GitHub

```python
def create_replay_memory(x, y, samples_per_class=400):
    selected = []

    for digit in TASK_A:
        indices = torch.where(y == digit)[0]

        # Selección reproducible de ejemplos de cada clase.
        generator = torch.Generator().manual_seed(SEED + digit)
        permutation = torch.randperm(
            len(indices), generator=generator
        )

        chosen = indices[permutation[:samples_per_class]]
        selected.append(chosen)

    selected = torch.cat(selected)

    return x[selected].clone(), y[selected].clone()


memory_x, memory_y = create_replay_memory(
    x_train_A,
    y_train_A,
    samples_per_class=400
)

print("Ejemplos guardados:", len(memory_y))
print("Clases en la memoria:", torch.unique(memory_y).tolist())
```

```python
# Entrenamiento inicial de A
print("=" * 45)
print("ENTRENAMIENTO DE LA TAREA A")
print("=" * 45)

train_model(
    replay_model,
    x_train_A,
    y_train_A,
    epochs=EPOCHS,
    lr=LEARNING_RATE
)

replay_after_A = {
    "A": evaluate(replay_model, x_test_A, y_test_A),
    "B": evaluate(replay_model, x_test_B, y_test_B)
}

print("\nResultados después de entrenar A:")
print(f"Precisión en A: {replay_after_A['A']['accuracy']:.2f}%")
print(f"Precisión en B: {replay_after_A['B']['accuracy']:.2f}%")

# Mezclamos los datos nuevos con la memoria de A
replay_train_x = torch.cat([x_train_B, memory_x], dim=0)
replay_train_y = torch.cat([y_train_B, memory_y], dim=0)

print("\n" + "=" * 45)
print("ENTRENAMIENTO DE LA TAREA B CON EXPERIENCE REPLAY")
print("=" * 45)

train_model(
    replay_model,
    replay_train_x,
    replay_train_y,
    epochs=EPOCHS,
    lr=LEARNING_RATE
)

replay_after_B = {
    "A": evaluate(replay_model, x_test_A, y_test_A),
    "B": evaluate(replay_model, x_test_B, y_test_B)
}
```

```
============================================
ENTRENAMIENTO DE LA TAREA A
============================================
Época 1/3 - pérdida: 0.6457
Época 2/3 - pérdida: 0.1383
Época 3/3 - pérdida: 0.0943

Resultados después de entrenar A:
Precisión en A: 98.54%
Precisión en B: 0.00%

============================================
ENTRENAMIENTO DE LA TAREA B CON EXPERIENCE REPLAY
============================================
Época 1/3 - pérdida: 1.8673
Época 2/3 - pérdida: 0.2988
Época 3/3 - pérdida: 0.1996

Resultados después de entrenar B:
Precisión en A: 89.86%
Precisión en B: 96.21%

============================================
RESUMEN FINAL DE EXPERIENCE REPLAY
============================================
...
B antes de B:  0.00%
A después de B: 89.86%
B después de B: 96.21%
Olvido en A: 8.68 puntos porcentuales
```

Código EWC

Enlace GitHub

```python
def estimate_fisher(model, x, y, max_samples=1000):
    model.eval()

    fisher = {
        name: torch.zeros_like(param)
        for name, param in model.named_parameters()
        if param.requires_grad
    }

    n = min(max_samples, len(y))
    indices = torch.randperm(len(y))[:n]

    for idx in indices:
        model.zero_grad(set_to_none=True)

        image = x[idx:idx + 1].to(device)
        label = y[idx:idx + 1].to(device)

        logits = model(image)
        log_probs = F.log_softmax(logits, dim=1)
        log_likelihood = log_probs[0, label.item()]

        log_likelihood.backward()

        for name, param in model.named_parameters():
            if param.requires_grad and param.grad is not None:
                fisher[name] += param.grad.detach().pow(2)

    for name in fisher:
        fisher[name] /= n

    return fisher
```

```python
ewc_model = new_model()

# 1. Estado inicial
ewc_initial_A = evaluate(ewc_model, x_test_A, y_test_A)
ewc_initial_B = evaluate(ewc_model, x_test_B, y_test_B)

print("=" * 45)
print("RESULTADOS INICIALES")
print("=" * 45)
print(f"Precisión A: {ewc_initial_A['accuracy']:.2f}%")
print(f"Precisión B: {ewc_initial_B['accuracy']:.2f}%")


# 2. Entrenamiento de la tarea A
print("\n" + "=" * 45)
print("ENTRENAMIENTO DE LA TAREA A")
print("=" * 45)

train_model(
    ewc_model,
    x_train_A,
    y_train_A,
    epochs=EPOCHS,
    lr=LEARNING_RATE
)

ewc_after_A = {
    "A": evaluate(ewc_model, x_test_A, y_test_A),
    "B": evaluate(ewc_model, x_test_B, y_test_B)
}

print("\nResultados después de entrenar A:")
print(f"Precisión A: {ewc_after_A['A']['accuracy']:.2f}%")
print(f"Precisión B: {ewc_after_A['B']['accuracy']:.2f}%")


# 3. Guardar parámetros aprendidos con A
old_params = {
    name: param.detach().clone()
    for name, param in ewc_model.named_parameters()
    if param.requires_grad
}
```

```python
fisher = estimate_fisher(
    ewc_model,
    x_train_A,
    y_train_A,
    max_samples=1000
)

fisher_values = torch.cat([
    values.detach().flatten()
    for values in fisher.values()
])

print(f"Media de Fisher: {fisher_values.mean().item():.8f}")
print(f"Máximo de Fisher: {fisher_values.max().item():.8f}")
print(f"Valores mayores que cero: {(fisher_values > 0).sum().item()}")
print(f"Total de valores: {fisher_values.numel()}")


# 5. Definir la penalización EWC
EWC_LAMBDA = 15000.0

def ewc_penalty():
    penalty = torch.zeros((), device=device)

    for name, param in ewc_model.named_parameters():
        if param.requires_grad:
            penalty += (
                fisher[name] *
                (param - old_params[name]).pow(2)
            ).sum()

    return (EWC_LAMBDA / 2) * penalty


def inspect_ewc():
    weighted_change = 0.0
    total_change = 0.0

    for name, param in ewc_model.named_parameters():
        if param.requires_grad:
            delta = param.detach() - old_params[name]
```

Código EWC

Enlace GitHub

```
            weighted_change += (
                fisher[name] * delta.pow(2)
            ).sum().item()

            total_change += delta.pow(2).sum().item()

    print(f"Cambio ponderado por Fisher: {weighted_change:.8f}")
    print(f"Cambio total de parámetros: {total_change:.8f}")
    print(f"Penalización EWC: {ewc_penalty().item():.8f}")


# 6. Diagnóstico antes de entrenar B
ewc_model.eval()

with torch.no_grad():
    logits = ewc_model(x_train_B[:512].to(device))
    loss_B = F.cross_entropy(
        logits,
        y_train_B[:512].to(device)
    )

print("\nDiagnóstico antes de entrenar B:")
print(f"Pérdida de clasificación B: {loss_B.item():.6f}")
print(f"Penalización EWC: {ewc_penalty().item():.6f}")
print(f"Lambda: {EWC_LAMBDA}")


# 7. Entrenamiento de la tarea B con EWC
print("\n" + "=" * 45)
print("ENTRENAMIENTO DE LA TAREA B CON EWC")
print("=" * 45)

train_model(
    ewc_model,
    x_train_B,
    y_train_B,
    epochs=EPOCHS,
    lr=LEARNING_RATE,
    extra_loss_fn=ewc_penalty
)
```

```
ewc_after_B = {
    "A": evaluate(ewc_model, x_test_A, y_test_A),
    "B": evaluate(ewc_model, x_test_B, y_test_B)
}

print("\nResultados después de entrenar B:")
print(f"Precisión A: {ewc_after_B['A']['accuracy']:.2f}%")
print(f"Precisión B: {ewc_after_B['B']['accuracy']:.2f}%")


# 8. Resumen final
olvido_A = (
    ewc_after_A["A"]["accuracy"]
    - ewc_after_B["A"]["accuracy"]
)

precision_promedio = (
    ewc_after_B["A"]["accuracy"]
    + ewc_after_B["B"]["accuracy"]
) / 2
```

```
============================================
RESULTADOS INICIALES
============================================
Precisión A: 0.74%
Precisión B: 19.46%

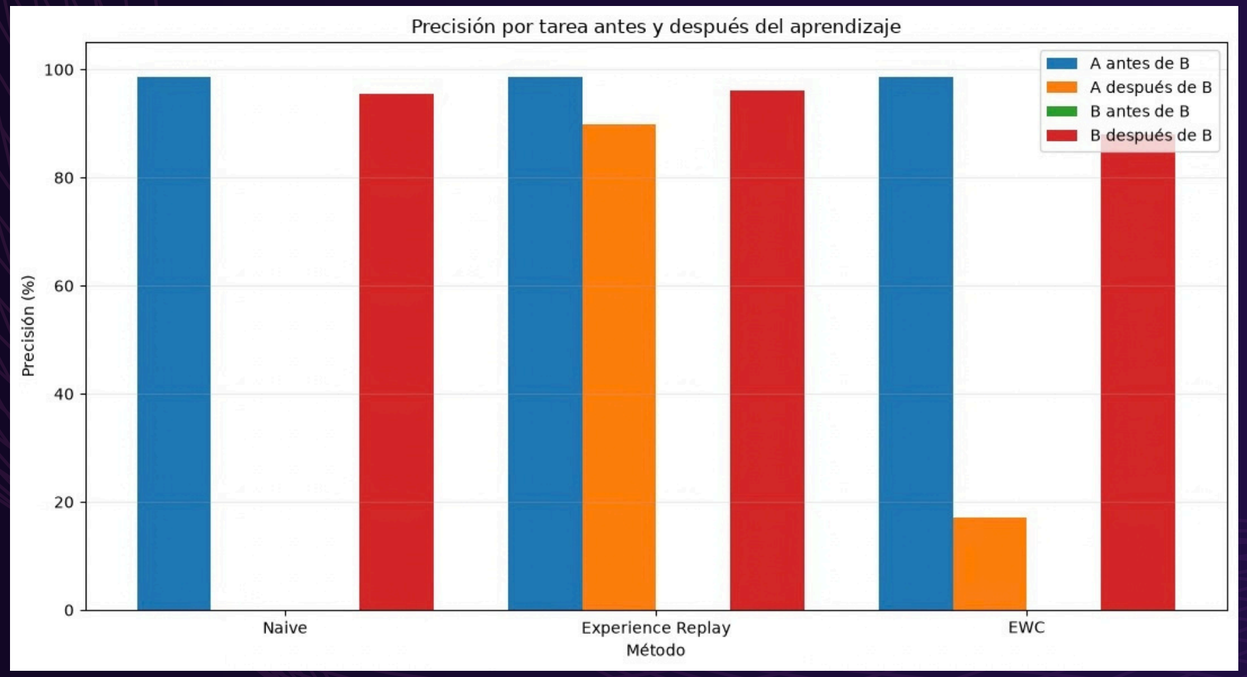
============================================
ENTRENAMIENTO DE LA TAREA A
============================================
Época 1/3 - pérdida: 0.6457
Época 2/3 - pérdida: 0.1383
Época 3/3 - pérdida: 0.0943

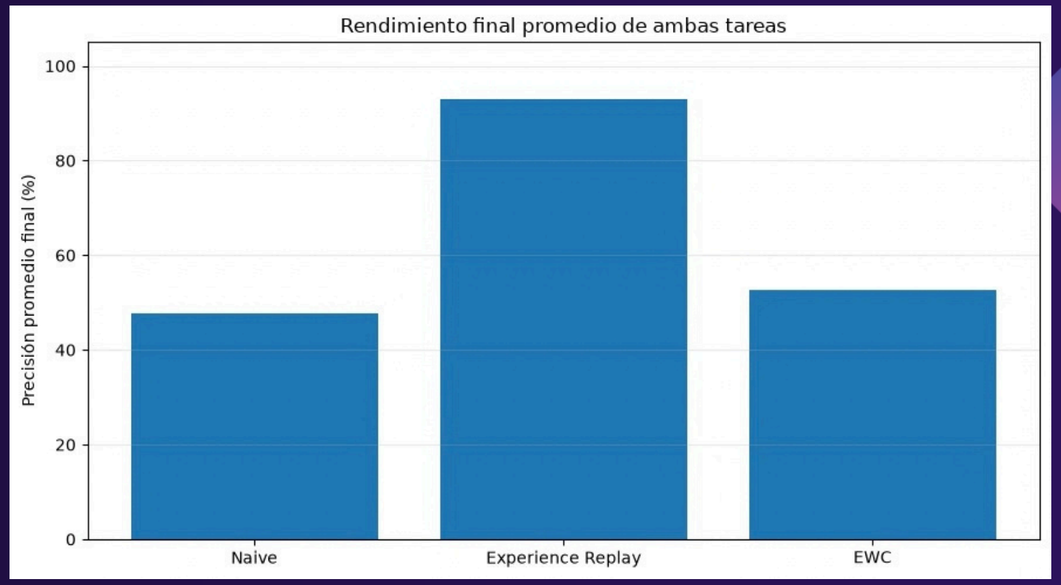
Resultados después de entrenar A:
Precisión A: 98.54%
Precisión B: 0.00%

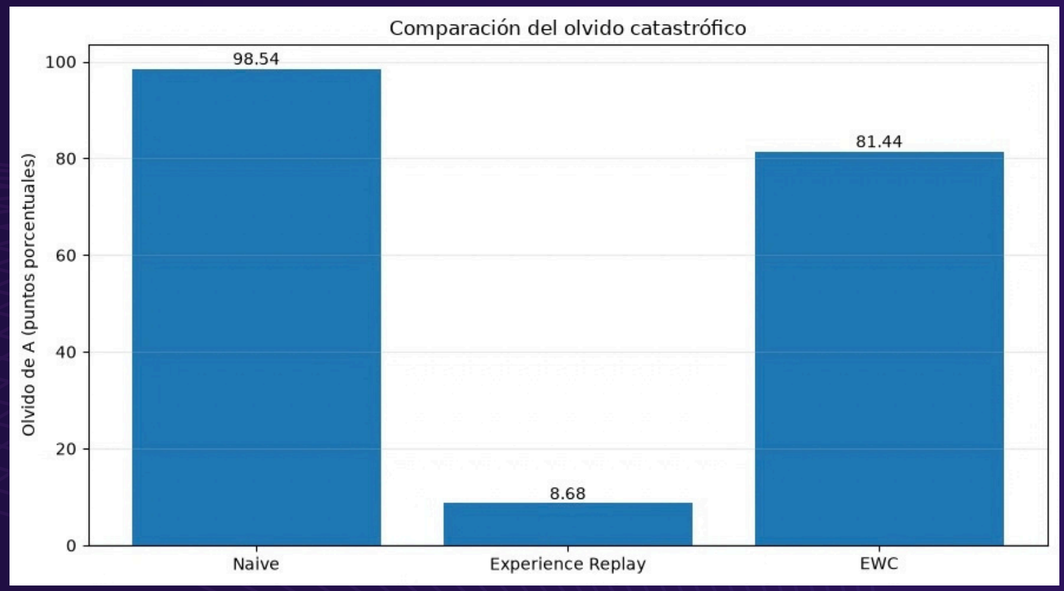
============================================
ESTIMACIÓN DE LA MATRIZ DE FISHER
============================================
Media de Fisher: 0.00046155
Máximo de Fisher: 2.14765072
Valores mayores que cero: 89706
Total de valores: 206922
...
Diagnóstico final de EWC:
Cambio ponderado por Fisher: 0.00001510
Cambio total de parámetros: 32.18415307
Penalización EWC: 0.11323697
```

Código
Resultados

Enlace GitHub







Aplicaciones **en la vida real**

- Modelos de lenguaje (LLMs)
  - El conocimiento del mundo cambia constantemente
  - Se actualizan en etapas: continual pre-training, continual fine-tuning y alineación
  - Reto: incorporar lo nuevo sin perder capacidades previas ni reentrenar desde cero
- Recomendación, finanzas y publicidad
  - Gustos, tendencias y comportamiento de usuarios evolucionan
  - El modelo debe seguir el flujo de datos sin olvidar patrones útiles
- Ciberseguridad y detección de anomalías
  - Aparecen nuevos tipos de ataque y de fraude
  - Se deben reconocer clases nuevas sin olvidar las conocidas
- Robótica
  - El robot aprende nuevas tareas de manipulación y exploración mientras opera
  - Reto: aprender de forma autónoma, con memoria y cómputo limitados
- Vehículos autónomos y drones
  - Nuevas ciudades, clima, señales y condiciones del entorno
  - Los modelos congelados no manejan bien distribuciones que cambian
- Salud
  - Incorporar nuevas enfermedades, equipos o datos de otros hospitales
  - La privacidad limita guardar datos de pacientes, lo que favorece métodos sin memoria de ejemplos (como EWC)

Continual Learning
Conclusiones

- El problema: al aprender secuencialmente, las redes sufren olvido catastrófico
- El reto de fondo: equilibrar estabilidad y plasticidad
- Lo que vimos en el experimento (MNIST, tareas A y B):

	A_antes_B	B_antes_B	A_despues_B	B_despues_B	Olvido_A_pp	Cambio_B_pp	Precision_promedio_final
Naive	98.54	0.0	0.00	95.47	98.54	95.47	47.74
Experience Replay	98.54	0.0	89.86	96.21	8.68	96.21	93.04
EWC	98.54	0.0	17.10	88.07	81.44	88.07	52.59

- Naive: olvido total
- Replay: el mejor resultado en este experimento, a cambio de guardar datos
- EWC: protege parcialmente sin guardar datos, pero paga con plasticidad y es sensible a λ y a Fisher
- No hay una estrategia universal: depende de la memoria, la privacidad y el cómputo disponibles
- Limitación: una sola corrida, dos tareas, escenario class-incremental
- Hacia dónde sigue: más tareas, combinar métodos (por ejemplo, EWC + replay) y aprendizaje continuo en LLMs y robótica